LAB

8

Operating Systems · FCIM / FAF · UTM · 2026-2027

The Same Calls, Inside Your Own OS

B **Track B: build your own small OS (xv6) · 90 minutes**

AT A GLANCE

Part 1	run, inside xv6	~30 min
Part 2	pingpong	~45 min
You submit	user/run.c and user/pingpong.c (and user/primes.c if you did it), and the Makefile with your UPROGS lines.	

Goal

In Lab 7 you ran fork, exec and wait on Linux. Today you run the same program inside xv6, the small OS you build this semester, and see that the calls are the same. Then you make two processes talk through pipes (pingpong).

Before you start

- Use your ~/xv6-riscv tree. Check that it builds and boots: make qemu, then Ctrl-A X to leave.
- Every new program is a file user/NAME.c plus one line $U/_NAME\ in the UPROGS list of the Makefile. The line starts with a TAB, and nothing may follow the backslash.

FILE Makefile

```
UPROGS=\
	$U/_cat\
	...
	$U/_zombie\
	$U/_run\
	$U/_pingpong\
```

Minimum to pass

Part 1, with its answers in notes.md. pingpong brings the rest of the grade. primes is optional and not graded.

PART 1 run, inside xv6 ~30 min

The same three steps as run.c in Lab 7, written for xv6: there is no glibc here, only the system calls of the kernel you compiled. The child becomes echo.

1.1 Create user/run.c with exactly this content:

FILE user/run.c

```
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child
int
main(int argc, char *argv[])
```

```
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                    // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                        // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                // step 2: become echo
    printf("exec failed\n");           // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);                       // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

1.2 Add $U/_run\ to UPROGS, then start xv6 and run it:

TERMINAL

```
make qemu
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

1.3 Make the child run ls. Change the two lines below, then make qemu again (Ctrl-A X first, to leave xv6):

CODE

```
    char *args[] = { "ls", 0 };
    exec("ls", args);                // step 2: become ls
```

1.4 Now exec a program that does not exist, for example exec("nosuchprog", args);, and run again.

Expected output (the end):

EXPECTED OUTPUT

```
exec failed
parent: child 8 ended with status 1
```

Observe (write in notes.md)

1. Write the three calls that run.c uses in xv6 and the three calls run.c used on Linux in Lab 7. What is the same?
2. Who prints "hello from the child": the code of run, or echo?
3. Why is the status 1 here and 127 on Linux? (Look at the exit line after exec in both programs.)

PART 2 pingpong ~45 min

Write user/pingpong.c. The parent sends one byte to the child through a first pipe. The child prints "<pid>: received ping" (its own pid), writes the byte back through a second pipe and exits. The parent reads it, prints "<pid>: received pong" and waits for the child. Complete the TODO lines:

FILE user/pingpong.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    // TODO: read 1 byte from p2c[0] into b
    printf("%d: received ping\n", getpid());
    // TODO: write the byte b into c2p[1]
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  // TODO: write 1 byte into p2c[1]
  // TODO: read 1 byte from c2p[0] into b
  printf("%d: received pong\n", getpid());
  wait(0);
  exit(0);
}
```

The calls you need: `read(fd, &b, 1)` and `write(fd, &b, 1)`. Add $U/_pingpong\ to UPROGS, then:

TERMINAL

```
make qemu
$ pingpong
10: received ping
9: received pong
$
```

(Your PIDs may differ. "ping" must come first, from the child.)

Observe (write in notes.md)

1. Why do you need TWO pipes, and not one for both directions?
2. Nothing in your code says "wait until the child has replied". Which call makes the parent wait?

Optional, not graded: primes

Write user/primes.c that prints the primes from 2 to 35 with a chain of processes: each stage reads its first number p (a prime, so it prints "prime p"), creates a pipe and a child, and forwards only the numbers not divisible by p. Close every pipe end you do not use, or you run out of descriptors.

FINAL Wrap-up and deliverable ~5 min

You submit:

- ☐ user/run.c and user/pingpong.c (and user/primes.c if you did it), and the Makefile with your UPROGS lines.
- ☐ notes.md: the answers to the "Observe" boxes.

Push to your course repository. The assistant builds your xv6 and runs run and pingpong.

Grading (out of 10):

Criterion	Points
Part 1: run works in xv6 (echo), answers in notes.md	3
Part 1: the ls and nosuchprog variants run	2
Part 2: pingpong prints ping then pong	3
Part 2: answers in notes.md	2
Total	10